Lecture 03.05: Match Case in Python (Structural Pattern Matching)

📌 Executive Summary

PEP 634 Structural Pattern Matching introduced in Python 3.10: match/case syntax, wildcards, guards, and sequence destructuring.

🔬 Theoretical Foundations & Principles

1. Structural Pattern Matching (PEP 634)

Python 3.10 introduced `match ... case`, a sophisticated pattern matching facility far superior to C-style `switch` statements.

2. Pattern Matching Capabilities

- **Literal Patterns:** Match exact scalar values.
- **Capture Patterns:** Bind matched values to variable names.
- **Wildcard Pattern (`_`):** Fallback matching any value without binding.
- **OR Patterns (`|`):** Combine multiple alternatives.
- **Guard Expressions (`if`):** Add arbitrary Boolean constraints to a pattern.
- **Sequence & Mapping Patterns:** Deconstruct data structures by shape.

3. Formal Pattern Predicate

A case matches target subject s if pattern $P(s) = \text{True}$ and guard $G(s) = \text{True}$:

$$\text{Action}(s) = \text{Exec}(B_i) \iff P_i(s) \wedge G_i(s) \wedge \left(\bigwedge_{j<i} \neg(P_j(s) \wedge G_j(s)) \right)$$

💻 Production Implementation & Code

```
def process_command(cmd: str) -> str:
    match cmd.strip().lower().split():
        case ["quit"] | ["exit"]:
            return "TERMINATING_ENGINE"
        case ["load", filename]:
            return f"LOADING_FILE: {filename}"
        case ["move", ("left" | "right" | "up" | "down") as direction, steps] if s
            return f"MOVING_{direction.upper()}_BY_{steps}_UNITS"
        case _:
            return "UNKNOWN_COMMAND"

print(process_command("load dataset.csv"))
print(process_command("move right 5"))
print(process_command("quit"))
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Detailed in respective algorithmic operations.
- **Auxiliary Space:** Memory bounds and CPython frame management.